GUARDIAN EYE

EDGE-FIRST EMERGENCY MONITORING

FULL SYSTEM TECHNICAL DOCUMENTATION

Multi-modal real-time monitoring

Vision, acoustic, and gesture fusion on the edge — with a realtime guardian dashboard, Telegram delivery, and optional GSM SMS.

Vision
YOLOv8 pose · fall rules

Audio
English + Malayalam keywords

Gesture
Hand-on-chest detection

Prepared by
Yedukrishna R
Guardian Eye project

Version 1.0 · October 2026
Repository: github.com/yedukrishna012/Guardian-Eye
MIT License

This document describes the Python edge engine and the Guardian Eye website in the same repository.

GUARDIAN EYE · TECHNICAL DOCUMENTATION

Contents

System architecture, timed decision rules, machine-learning models, Firestore synchronization, the Guardian Eye website, deployment, security, and known limitations.

1. System Overview
2. Architecture and Key Files
   - Edge engine
   - Website
3. Edge Processing Components
   - 3.1 main.py — process orchestration
   - 3.2 fall_process.py and fall.py — vision and gestures
   - 3.3 voice_process.py — acoustic detection
   - 3.4 decision_engine.py — timed sensor fusion
   - 3.5 Alert channels
4. Machine Learning Models
   - YOLOv8 Pose (yolov8n-pose.pt)
   - YOLOv8 Detection (yolov8n.pt)
   - Vosk speech model (models/en)
   - Optional Gemini vision verification
5. Firestore Synchronization
6. Guardian Eye Website
   - Public experience
   - Read-only demo (/demo)
   - Authenticated dashboard
   - Camera stream security
   - Environment configuration
   - PWA
7. Setup Summary
8. Security Notes
9. Known Limitations
10. License and Credits

1. System Overview

Guardian Eye is an edge-first emergency monitoring system. It watches a person through a camera and a microphone, runs fall and gesture detection plus English keyword spotting on the local device, and raises an alert only when a timed decision rule is satisfied. Malayalam keyword recognition uses Google Speech when connectivity allows.

The system has two parts in one repository:

- **Python edge engine** — sensor capture, local vision and English speech inference, the decision state machine, Firebase writes, Telegram messaging, and optional GSM SMS.
- **Guardian Eye website** (`website/`) — the public project site, an interactive rule explorer, a read-only demo, and the authenticated guardian dashboard.

The edge engine keeps detecting when the network is unavailable. Firebase, Telegram, and GSM are independent response paths rather than hard dependencies for detection.

---

2. Architecture and Key Files

Edge engine

File	Responsibility
`main.py`	Starts the vision and voice processes, reads their event queues, runs the decision engine, and triggers GSM SMS when the shared emergency flag rises
`fall_process.py`	Camera pipeline, YOLOv8 pose inference, gesture evaluation, annotated frames, and the local MJPEG stream
`fall.py`	Fall heuristics, temporal state machine, furniture-aware suppression, and optional Gemini verification
`gesture.py`	Keypoint geometry for hand-on-head, hand-on-chest, raised hands, and crossed arms
`voice_process.py`	Continuous microphone capture, English Vosk recognition, Malayalam recognition, and keyword events
`decision_engine.py`	Timed multi-signal rules, Firestore writes, and Telegram dispatch
`telegram_alert.py`	Telegram Bot API delivery
`gsm.py`	SIM800L serial SMS path on `/dev/serial0`
`device_config.example.json`	Pairing template; copy to the git-ignored `device_config.json`
`service_account.json`	Firebase Admin SDK credentials; git-ignored

`gesture_process.py` is an alternative standalone gesture process and is not started by the current `main.py`; the active pipeline evaluates gestures inside `fall_process.py`.

Website

```
website/
├── src/pages/          Landing, Login, Signup, Demo, Dashboard, Statistics, Profile
├── src/components/     Shared UI, camera dialog, emergency overlay, incident explorer
│   └── landing/        Lazy 3D hero, decision lab, simulated dashboard preview
├── src/contexts/       Firebase Auth provider and hook
├── src/demo/           Simulated incidents for /demo
├── src/utils/          Formatting, CSV, clipboard, auth error messages
├── src/firebase.js     Environment-based Firebase initialization
├── firestore.rules     Browser access rules
└── vite.config.js      Vite + PWA manifest
```

The site is a React 19 single-page application built with Vite, Tailwind CSS, Framer Motion, React Three Fiber, and Firebase. It is installable as a PWA and included in the same repository as the edge engine.

---

3. Edge Processing Components

3.1 `main.py` — process orchestration

Computer vision and microphone processing run in separate OS processes so one pipeline cannot block the other:

1. `main.py` creates a shared `event_queue` and a shared emergency flag (`multiprocessing.Value`).
2. It starts `run_fall_process()` and `run_voice_process()`.
3. It continuously consumes fall, gesture, and voice events and passes them to `EmergencyDecisionEngine`.
4. When the engine sets the shared emergency flag, `main.py` calls `send_sms()` for the hardware alert path.

The engine latches after the first confirmed emergency for the lifetime of the process. Restart `main.py` to re-arm it.

3.2 `fall_process.py` and `fall.py` — vision and gestures

The camera pipeline tries, in order:

1. Picamera2 on Raspberry Pi OS,
2. a GStreamer `libcamerasrc` pipeline,
3. OpenCV camera indices with MJPG and YUYV fallbacks.

Frames are resized and passed to `SimpleHighAccuracyFallDetector`. The detector uses YOLOv8 pose keypoints to estimate torso angle and body aspect ratio. A fall must be consistently suspected for several frames before it enters `FALL_DETECTED`.

Two suppression mechanisms reduce false positives:

- **Furniture-aware check** — when a fall is highly suspected, a YOLOv8 object detector looks for COCO classes 56 (chair), 57 (couch), and 59 (bed). A horizontal person centered inside a furniture box is treated as resting.
- **Optional Gemini verification** — if `gemini_api_key` is present in `device_config.json`, an anomaly frame can be sent to a Gemini vision model for a YES/NO check. If the model is unavailable or errors, the detector falls back to its pose-based decision.

The same camera loop evaluates gestures from the pose keypoints. `gesture.py` uses wrist, nose, shoulder, and chest-region geometry to identify hands on chest, hands on head, raised hands, and crossed arms. `fall_process.py` forwards state changes for hand-on-chest and other gestures to the decision engine.

The MJPEG stream is served by Flask on port `5000` at `/video_feed`. An annotated preview is written to a shared frame buffer for the browser.

3.3 `voice_process.py` — acoustic detection

The voice process captures 16 kHz mono audio in 1.5-second blocks and adapts the input rate when the microphone does not support 16 kHz natively. It uses:

- **English:** a local Vosk model at `models/en` for low-latency keyword spotting.
- **Malayalam:** Google Speech Recognition (`ml-IN`) on sampled chunks when CPU load allows.

Detected keywords are pushed to the event queue as `type: "voice"` with the keyword, language, and timestamp. The decision engine debounces duplicate keywords within one second and retains only the last 20 seconds of voice events.

3.4 `decision_engine.py` — timed sensor fusion

The engine evaluates five rules in priority order. The first matching rule wins.

Priority	Condition	Stored reason
1	Fall state persists for **≥ 10 seconds**	`Fall detected for ≥ 10 seconds`
2	Fall state persists for **≥ 5 seconds** and at least one voice keyword occurred in the 20-second window	`Fall ≥ 5 seconds + voice emergency keyword`
3	**≥ 3 voice keywords** within 20 seconds	`Voice emergency keyword repeated ≥ 3 times within 20 seconds`
4	Hand-on-chest gesture persists for **≥ 10 seconds**	`Hand on chest detected for ≥ 10 seconds`
5	Hand-on-chest is active and at least one voice keyword occurred	`Hand on chest + voice emergency keyword`

The engine fires once, sets the emergency flag, writes the event, and dispatches Telegram. It does not use a severity field.

3.5 Alert channels

- **Firestore** — the dashboard receives events through a realtime listener.
- **Telegram** — `telegram_alert.py` sends the reason, time, and paired user ID to every configured chat ID.
- **GSM SMS** — `main.py` calls `send_sms()` when the emergency flag rises. The SIM800L path uses AT commands on `/dev/serial0` at 9600 baud. It is an additional hardware channel, not a check that Wi-Fi has failed; if the module or SIM is unavailable, the attempt fails while Firebase and Telegram continue.
- **Browser overlay** — the authenticated dashboard shows a full-screen alert while it is open. Guardian Eye does **not** send browser push notifications. Telegram and optional GSM SMS are the out-of-app channels.

---

4. Machine Learning Models

YOLOv8 Pose (`yolov8n-pose.pt`)

The pose model identifies 17 body keypoints. `fall.py` combines torso angle from shoulders to hips, bounding-box aspect ratio, ankle position, and temporal consistency to estimate fall confidence. Gesture recognition reuses the same keypoints.

YOLOv8 Detection (`yolov8n.pt`)

The standard detector is used conditionally for furniture-aware suppression. It is loaded at startup alongside the pose model. The furniture check only runs when a fall is already suspected, which limits its CPU cost.

Vosk speech model (`models/en`)

The English Vosk model is downloaded separately because of its size. Without it, the vision pipeline still runs but English voice detection is skipped.

Optional Gemini vision verification

When configured, Gemini acts as a second opinion on a suspected fall. It is optional, requires network access and an API key, and is not the primary detector.

---

5. Firestore Synchronization

A confirmed emergency is written inline in `EmergencyDecisionEngine.trigger_emergency()`:

```
users/{user_id}/emergencies/{auto_id}
  reason:    string   // the exact rule reason
  status:    "active"
  timestamp: Firestore server timestamp
```

The Firebase Admin SDK uses `service_account.json` and bypasses browser security rules. The paired `user_id` comes from `device_config.json`. There is no severity field and no separate named push function.

The dashboard subscribes with `onSnapshot` to:

```
users/{uid}/emergencies  ordered by timestamp descending
```

When a guardian acknowledges an event, the browser updates only `status` to `"resolved"`. The event remains in history. The full-screen overlay hides after five minutes for an unacknowledged active alert, but the event stays visible with an acknowledge action until it is resolved.

The browser rules in `website/firestore.rules` allow a signed-in user to:

- read their own emergency history,
- create a test event with only `reason`, `status: "active"`, and a server timestamp,
- change only `status` to `"resolved"`.

Deletes and all other access are denied.

---

6. Guardian Eye Website

Public experience

- Interactive 3D hero built with lazy-loaded React Three Fiber.
- Decision Lab that lets a visitor toggle simulated fall, voice, and chest-gesture inputs and see which rule would confirm first.
- Simulated dashboard preview with a 10-second fall countdown.
- Statistics preview, PWA install prompt, reduced-motion support, and accessible dialogs.

Read-only demo (`/demo`)

The demo uses `src/demo/mockData.js` and the same shared UI components as the real dashboard. A visitor can trigger a simulated emergency overlay, acknowledge it, open the camera dialog, and explore incident statistics with filters and CSV export. No Firebase credentials are required, no event is written, and no Telegram or SMS message is sent.

Authenticated dashboard

- Email/password sign-in and password reset through Firebase Authentication.
- Realtime Firestore listener with an empty, loading, and error state.
- Full-screen emergency overlay with acknowledge and camera actions.
- Stale-alert banner after five minutes.
- Test-alert button that writes a single event through the Firestore rules.
- Statistics with date/status/search filters, daily activity chart, signal breakdown, CSV export, and pagination.
- Profile page with account metadata, pairing ID, password reset, and sign-out.
- Camera dialog with device address, optional token, retry, fullscreen, and mixed-content guidance.

Camera stream security

The stream is intended for a trusted LAN.

- `CAMERA_STREAM_TOKEN` (optional) requires `?token=...` or a Bearer token on `/video_feed`.
- `CORS_ORIGINS` (optional) restricts browser origins; unset keeps the original permissive behavior.
- The dashboard stores the device address in local storage and the token in session storage.
- An HTTPS dashboard cannot load an HTTP camera stream. Use LAN HTTP or an HTTPS reverse proxy/tunnel.
- Port `5000` should not be exposed to the public internet.

Environment configuration

Firebase values live in `website/.env.local` and are never committed:

```
VITE_FIREBASE_API_KEY=
VITE_FIREBASE_AUTH_DOMAIN=
VITE_FIREBASE_PROJECT_ID=
VITE_FIREBASE_STORAGE_BUCKET=
VITE_FIREBASE_MESSAGING_SENDER_ID=
VITE_FIREBASE_APP_ID=
```

`VITE_CAMERA_STREAM_TOKEN` is optional and, like every Vite variable, is visible in the browser bundle.

PWA

`vite-plugin-pwa` generates a service worker and a web app manifest with 192 px, 512 px, and maskable icons. The manifest includes shortcuts for the dashboard and demo. Browsers that support installation show an in-app install button; installed windows run standalone.

---

7. Setup Summary

1. Install Python dependencies from `requirements.txt` (desktop) or `requirements_pi.txt` (Pi).
2. Download the Vosk English model into `models/en`.
3. Copy `.env.example` to `.env` and set Telegram/GSM values as needed.
4. Copy `device_config.example.json` to `device_config.json` and set the Firebase UID.
5. Place the Firebase Admin SDK key in `service_account.json`.
6. Start the edge engine with `python main.py`.
7. In `website/`, run `npm ci`, copy `.env.example` to `.env.local`, fill the Firebase web values, deploy `firestore.rules`, and run `npm run dev`.

Full setup details are in `README.md` and `website/README.md`.

---

8. Security Notes

- `service_account.json`, `device_config.json`, `.env`, and `website/.env.local` are git-ignored.
- Firebase web configuration is not secret, but Firestore access must be protected by rules and Authentication.
- The camera stream is not encrypted by the Python server. Keep it on a trusted network.
- Never expose port `5000` through a router or public reverse proxy without authentication, TLS, and rate limiting.
- Rotate any credential that was ever committed or shared.

---

9. Known Limitations

- The decision engine latches after the first emergency until the process restarts.
- Browser alerts appear only while the dashboard is open; there are no Web Push notifications.
- Malayalam recognition uses a network-dependent Google Speech API call and is skipped under CPU pressure.
- The camera dialog cannot bypass browser mixed-content blocking.

- The optional Gemini verification model name is fixed in `fall.py`; availability depends on the Gemini API version and account access.
- The system is a college project, not a certified medical or emergency-response device.

---

10. License and Credits

Released under the MIT License. See `LICENSE`.

Copyright (c) 2026 Yedukrishna R (@yedukrishna012)

Built as a college project by **Yedukrishna R**, **Jubin T Joseph**, **Muhammed Nabeel Sha A**, and **Rishijith Karthikeyan M**. The web dashboard was originally contributed by **jubin217**; see `NOTICE`.